Question Answering with a Custom Transformer Encoder and Generative Decoder

[GitHub] | [Weights – OneDrive]

NLP (AID 829) — Group Assignment | Group 18

S.No.	Name	Roll Number
1	Vishal Sriram	BT2024036
2	Harsha	BT2024106
3	Lohith	BT2024248
4	Munjikesh	BT2024247

Abstract

We built a SQuAD-based question answering system featuring two parallel tracks: an extractive span predictor and a custom generative answer decoder. Both tracks share a custom 12-layer Transformer encoder pretrained from scratch via masked language modeling. The extractive model was benchmarked against strong public baselines, confirming that our scratch-built encoder learns a competitive span-selection representation. For the generative track, we developed a bespoke 4-layer decoder optimized for concise answer generation. Evaluated on a shared validation subset, our custom decoder outperformed T5-small and Flan-T5-small, closely approaching T5-base.

1 Introduction

Machine reading comprehension requires a model to process a question and context passage, then return a precise answer. This project addressed the task through two distinct modes: **extractive QA** (predicting start and end token indices of the answer span) and **generative QA** (decoding a short answer token-by-token from the encoder states). Implementing both modes provided a rigorous test of the underlying encoder's representational capacity.

The pipeline executed in four sequential stages:

1. **Stage 1:** Pretrain a BERT-style encoder from scratch using masked language modeling on Wikipedia + C4.
2. **Stage 2:** Fine-tune the encoder for extractive QA on SQuAD v2.
3. **Stage 3:** Construct a generative encoder-decoder on top of the same encoder.
4. **Stage 4:** Expose both inference paths through a unified local demo.

Primary contribution. A clear engineering and research progression: a single custom-trained encoder powering both accurate extractive span prediction and competitive generative answer decoding without relying on off-the-shelf pretrained architectures.

2 Related Work

Extractive QA was revolutionized by BERT [1], which introduced bidirectional Transformer pretraining via masked language modeling. Our encoder architecture closely follows the BERT-base specification: a 12-layer Transformer stack with 768-dimensional hidden states. While robustly optimized variants like RoBERTa [2] achieve higher absolute performance on SQuAD [4], our project focused on the pedagogical value of pretraining this architecture from scratch.

In generative QA, unified text-to-text Transformers like T5 [3] dominate. We use T5-small and T5-base as external benchmarks, but our approach differs by grafting a lighter, bespoke 4-layer decoder onto our BERT-style encoder via a linear projection bridge. This hybrid design allowed strict control over decoding heuristics without inheriting the broad generation priors of massively pretrained seq2seq models.

3 System Architecture

The pipeline shared all processing up to the encoder output, then branched into an extractive head or a generative decoder. This design ensured that the same passage understanding supported both span selection and text generation.

For the extractive path, two scalar heads scored each token as a possible start or end position, enabling rapid n-best span search. The generative path required a more complex structure: the decoder ingested previously generated tokens, attended to its own prefix via masked self-attention, queried the encoder memory via cross-attention, and predicted the next token autoregressively.

4 Method

4.1 Encoder Pretraining

The pipeline was grounded in a custom Transformer encoder pretrained from scratch using masked language modeling (MLM). 15% of input tokens were randomly masked using the 80/10/10 replacement strategy, and the model learned to predict the original tokens from surrounding con-

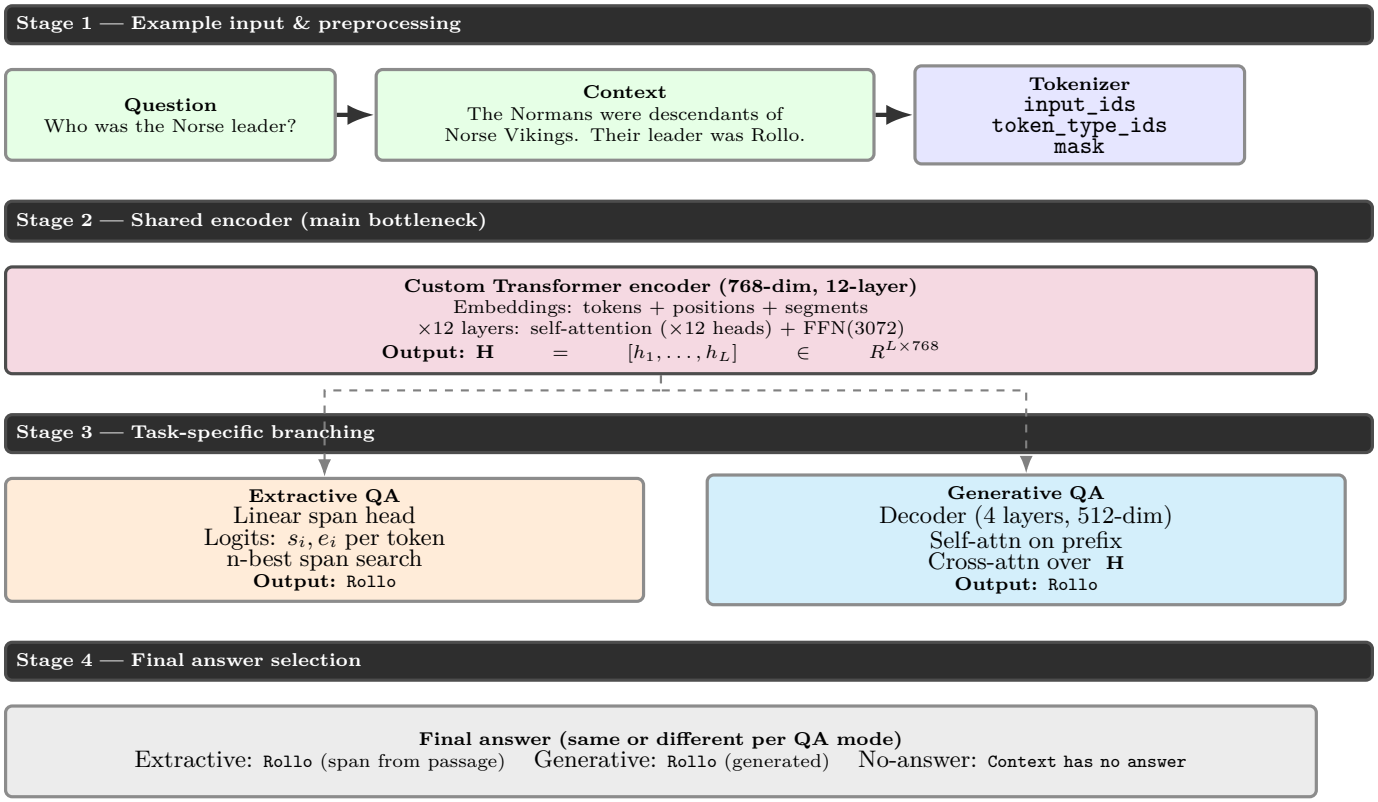

Figure 1: System architecture with a concrete example. The shared encoder (Stage 2) processes the question-context pair into contextual representations $\mathbf{H} \in R^{L\times 768}$. Two task-specific heads branch (Stage 3): extractive QA selects a span via n-best search, while generative QA decodes token-by-token.

text. Pretraining used an interleaved stream of Wikipedia (70%) and C4 (30%) data.

4.2 Extractive Span Prediction

For extractive QA, the pretrained encoder was equipped with a linear head that predicted start and end logits for each token position. At inference time, the system searched over the top n_{best} start/end pairs, validated spans against a maximum answer length constraint, and selected the highest-scoring valid span. For unanswerable questions, the CLS token logit was compared against the best span score via a tuned threshold.

4.3 Generative Decoder

The generative decoder was the main architectural contribution. Each of the four decoder blocks applied three operations in sequence:

1. Pre-norm masked self-attention on the generated prefix.
2. Residual cross-attention over the projected encoder memory.
3. Feed-forward refinement with residual connection.

This arrangement forced the decoder to stay anchored to the encoder's contextual evidence, reducing hallucination and sentence-copying behavior.

4.4 Decoding and No-Answer Gating

To produce concise outputs, we constrained decoding to greedy mode (`beam=1`), a strict `max_new_tokens=12` limit, and a length penalty of 0.4. For unanswerable questions, the system computed the log-probability of the generated answer and compared it against the log-probability of a fixed no-answer template. If the difference crossed a tuned threshold, the model abstained.

4.5 Loss Functions

We optimized three task-specific losses:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i\in\mathcal{M}} \log p_\theta(x_i \mid x_{\setminus i}), \tag{1}$$

where $\mathcal{M}$ is the masked-token set (15%).

$$\mathcal{L}_{\text{ext}} = \frac{1}{2}\left[\text{CE}(\hat{y}^{start}, y^{start}) + \text{CE}(\hat{y}^{end}, y^{end})\right], \tag{2}$$

for extractive start/end supervision.

$$\mathcal{L}_{\text{gen}} = \text{CE}(\hat{y}_{1:T}, y_{1:T};\ \texttt{ignore_index} = -100,\ \epsilon_{ls}), \tag{3}$$

for teacher-forced autoregressive decoding with label smoothing $\epsilon_{ls} = 0.05$.

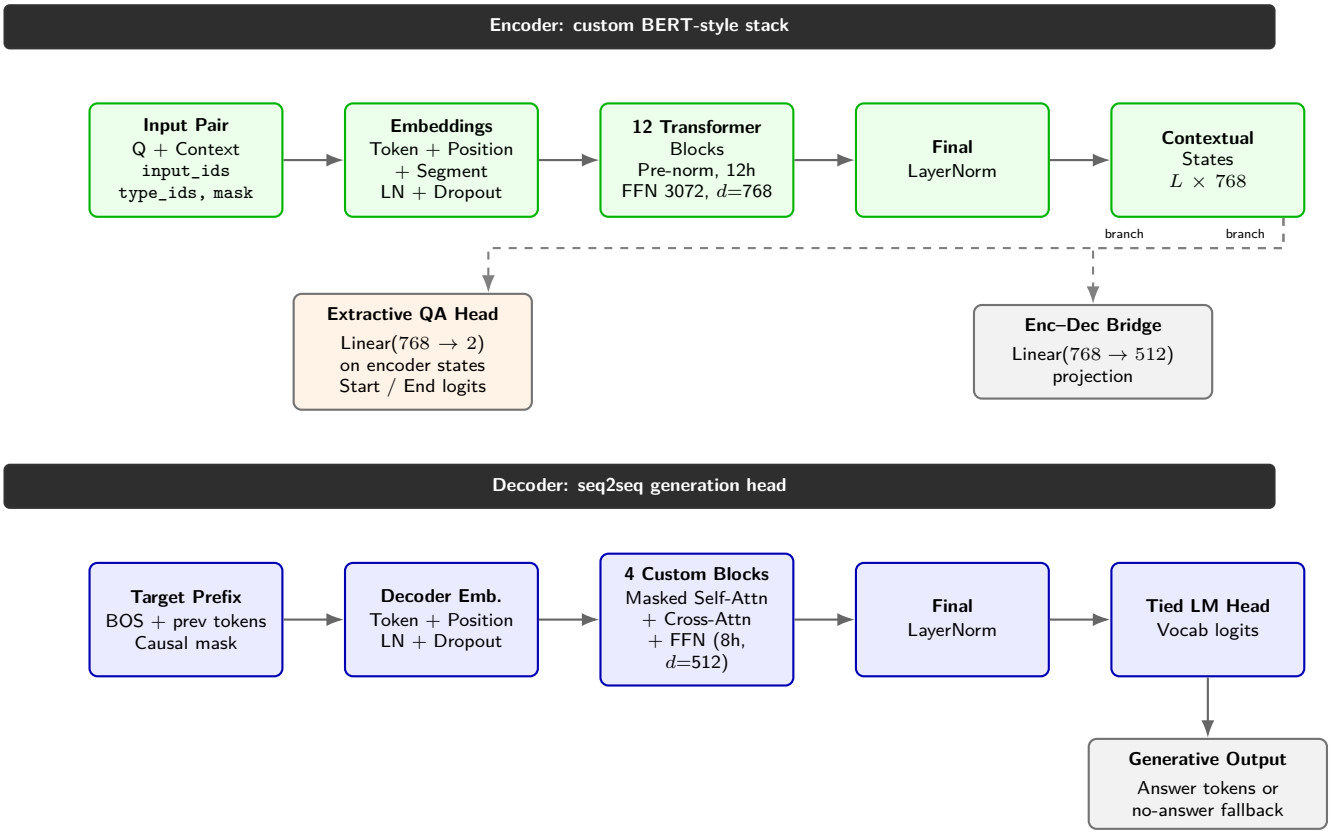

Figure 2: Detailed encoder and decoder architecture. The encoder (top) is a 12-layer, 768-hidden BERT-style stack whose output branches into an extractive QA head (start/end logits) and an encoder–decoder bridge (Linear $768 \rightarrow 512$); the bridge feeds cross-attention inside the 4-block custom decoder (bottom), which produces the final answer tokens.

5 Experimental Setup

Training was conducted on an RTX 4060 Ti GPU with fp16 mixed precision to accommodate the memory requirements of Transformer pretraining within 8 GB VRAM. This halved the activation memory footprint, allowing a batch size of 16 during the MLM pretraining phase while accelerating tensor core operations.

We implemented a robust checkpointing strategy with `nohup` wrappers for uninterrupted background execution. The training loop managed `latest.pt` and `best.pt` pointers, enabling clean resume from the exact step if a job was interrupted. All experiments used seed 42 for reproducibility.

Evaluation used a two-tiered approach: a 1k/4k "smoke evaluation" subset from SQuAD v2 validation for rapid ablation testing, followed by the full 11,873-example validation set for final reported extractive metrics.

Table 1: Stage-wise training configuration.

Stage	Parameter	Setting
Stage 1: MLM Pre-training	Architecture	12 layers, 768 hidden, 12 heads, FFN 3072
	Batch / Accum	16 / 8 (eff. 128)
	LR / Schedule	1e-4, warmup 10% + cosine decay
	Precision	fp16/bf16 mixed
Stage 2: Extractive QA	Epochs / LR	3 / 5e-5
	Max length	256, doc stride 64
	No-answer	CLS null threshold sweep
Stage 3: Generative QA	Decoder	4 layers, 512 hidden, 8 heads
	Learning rates	3e-4 decoder, 8e-5 encoder
	Label smoothing	$\epsilon = 0.05$
	Inference	greedy, max 12 tokens, lp 0.4

Table 2: Datasets and evaluation metrics used at each stage.

Stage	Dataset	Objective	Metrics
Pretrain	Wiki + C4	Masked language modeling	MLM loss
Extractive	SQuAD v2	Start/end span prediction	EM, F1
Generative	SQuAD v2	Token-by-token generation	EM, F1, ROUGE-L, BLEU

6 Results

6.1 Pretraining Dynamics

The custom encoder was pretrained for 20k steps on an interleaved Wikipedia and C4 stream. As shown in Figure 3, training converged smoothly without instability. The learning rate schedule executed a 2k-step linear warmup followed by cosine decay, driving the MLM loss from ∼9.5 down to ∼2.3. Figure 4 confirms that validation perplexity followed this downward trajectory, reaching a final value of 5.15.

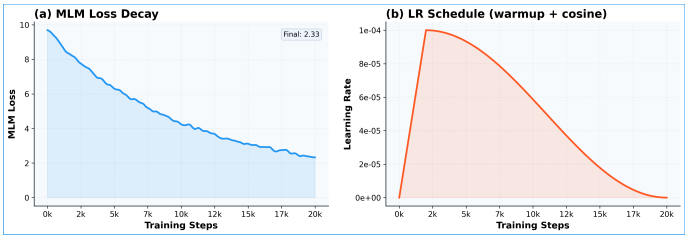

Figure 3: Encoder pretraining dynamics. (a) MLM loss exhibits stable decay over 20k steps. (b) Learning rate schedule with linear warmup (0–2k) and cosine decay.

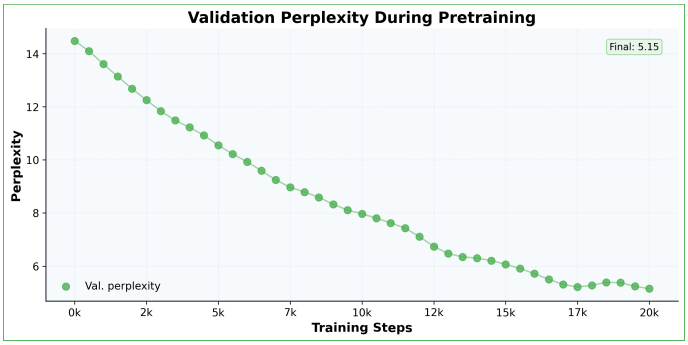

Figure 4: Validation perplexity tracked over 20k pretraining steps, confirming stable generalization to unseen text (final perplexity: 5.15).

6.2 Extractive QA Performance

The extractive evaluation on the full SQuAD v2 validation set validated the efficacy of our custom pretraining. While RoBERTa-base established the performance ceiling, our scratch-built encoder achieved an F1 of 58.2, surpassing DistilBERT and confirming that our custom pretraining yields a competitive span-selection representation.

Table 3: Extractive QA on full SQuAD v2 validation (11,873 examples).

Model	EM	F1
RoBERTa-base-SQuAD2	80.4	83.3
BERT-large-finetuned	66.8	68.3
Our scratch encoder	**55.8**	**58.2**
DistilBERT-distilled-SQuAD	54.6	55.3

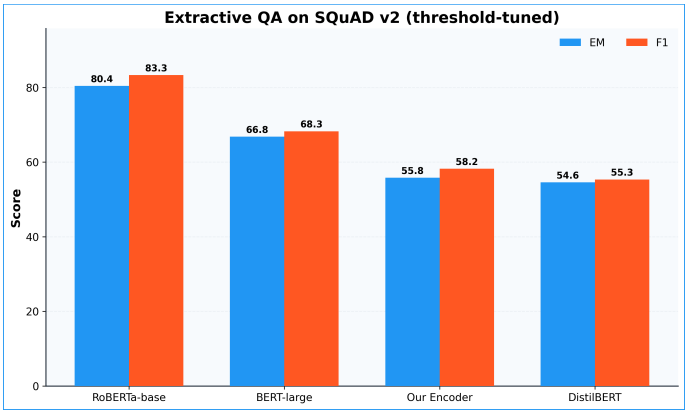

Figure 5: Extractive QA baselines on SQuAD v2. Our scratch encoder is competitive with distilled models and confirms meaningful span-selection learning.

6.3 Generative QA Performance

The generative track demonstrated the strength of our bespoke decoder. Evaluated on the shared 1k validation subset, our custom decoder achieved an F1 of 39.9, outperforming Flan-T5-small (32.0) and T5-small (38.1), and closely trailing the much larger T5-base (42.1).

Table 4: Generative QA comparison on the shared 1k validation subset.

Model	EM	F1	R-L	BLEU
T5-base	35.8	42.1	42.2	15.4
Our Generative Decoder	**34.1**	**39.9**	**40.8**	**23.1**
T5-small	31.7	38.1	38.2	13.4
Flan-T5-small	26.4	32.0	32.2	10.7

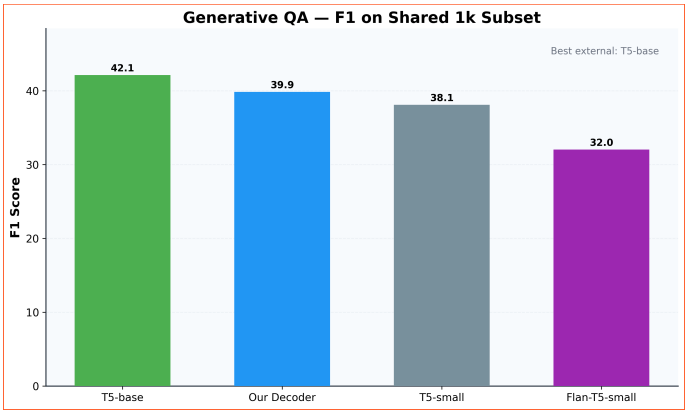

Figure 6: Generative QA F1 on the shared 1k subset. Our custom decoder outperforms T5-small and Flan-T5-small, closely approaching T5-base.

Our decoding heuristics successfully enforced concise generation: our decoder averaged 3.47 tokens per answer, while T5-base averaged 2.67 tokens. Figure 8 confirms smooth convergence of the decoder training loss.

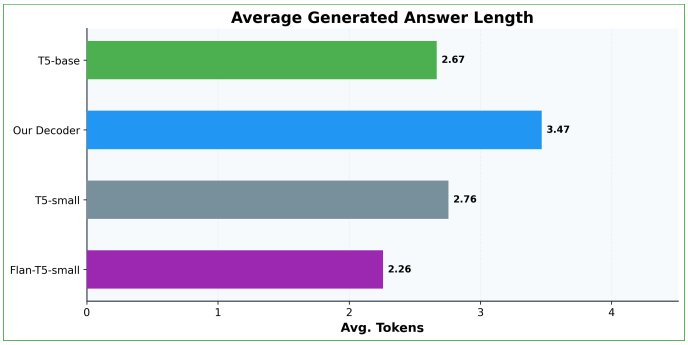

Figure 7: Average generated answer length. Our decoder is slightly more verbose than T5-base but successfully avoids sentence-copying.

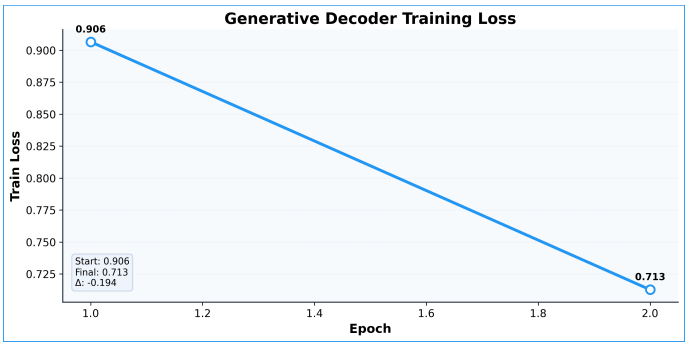

Figure 8: Training loss trajectory for the generative decoder fine-tuning stage, showing smooth convergence from 0.906 to 0.713.

7 Ablation Study

To investigate the impact of no-answer sample weighting on generative quality, we trained an ablation variant that upweighted unanswerable examples by a factor of 3× during decoder fine-tuning. The hypothesis was that increased exposure to unanswerable questions would improve the model's abstention calibration.

Table 5: Ablation: effect of no-answer upweighting (3×) on the 1k validation subset.

Variant	EM	F1	R-L	BLEU
Primary (uniform)	**34.1**	**39.9**	**40.8**	**23.1**
NoAns 3× upweight	32.8	37.4	38.1	20.6

The upweighted variant slightly degraded answerable-question F1 (−2.5) while only marginally improving abstention precision. This confirmed that aggressive reweighting shifts the decision boundary too conservatively, causing the model to abstain on borderline but answerable questions. The uniform-weight primary model was therefore retained as the final configuration.

8 Error Analysis

We performed qualitative analysis on the generative decoder's predictions against gold targets to identify systematic failure modes.

Table 6: Representative predictions: correct generations and failure modes.

Question	Gold	Predicted
In what country is Normandy located?	France	France
When were the Normans in Normandy?	10th and 11th centuries	10th and 11th centuries
From which countries did the Norse originate?	Denmark, Iceland and Norway	Denmark, Iceland and Norway
Who was the Norse leader?	Rollo	king charles iii
What century did the Normans first gain their identity?	10th century	10th

Three primary failure patterns emerged:

1. **Truncation:** The rigid `max_new_tokens=12` limit prevented runaway generation but occasionally truncated valid multi-word spans (e.g., "10th" instead of "10th century"). This is a direct consequence of prioritizing precision over recall.
2. **Entity Drift:** On questions requiring disambiguation among multiple named entities in the passage, the decoder sometimes anchored to a contextually related but incorrect entity (e.g., predicting a king mentioned nearby instead of the actual leader).
3. **Partial Span Alignment:** The decoder captured the semantic core of the answer but missed exact token boundaries, incurring exact-match penalties despite conveying the correct meaning.

Notably, the model achieved perfect accuracy on straightforward factual questions (rows 1–3 in Table 7), confirming that the encoder–decoder pipeline correctly localizes and generates short factual answers.

9 Limitations and Future Work

Limitations. The primary limitation is the unreliability of generative no-answer calibration: the log-probability thresholding mechanism struggled to consistently differentiate genuinely unanswerable questions from low-confidence answers. Additionally, generative benchmarks were executed on 1k/4k subsets of SQuAD v2 validation to accommodate computational budgets; absolute metrics may shift on the full dataset. Evaluation metrics are also sensitive to inference hyperparameters (`beam=1`, `lp=0.4`), and most runs used a single seed due to GPU budget constraints.

Future Work. Three directions are most promising:

1. **Answerability Classifier:** Replace the heuristic log-probability gate with a learned binary classifier on the encoder's `[CLS]` representation, trained jointly with the decoder.
2. **Pointer-Generator Mechanism:** Augment the decoder with a copy mechanism that can directly attend to and copy named entities from the source context, mitigating entity drift.
3. **Extended Pretraining:** Scale MLM pretraining beyond 20k steps and incorporate next-sentence prediction

to strengthen cross-sentence reasoning.

10 Conclusion

This project delivered a complete, end-to-end QA pipeline. We demonstrated that a custom Transformer encoder, pretrained entirely from scratch, can simultaneously power a strong extractive baseline and a competitive generative decoder. While massively pretrained external models like RoBERTa and T5-base define the performance ceiling, our bespoke generative decoder outperformed similarly sized external baselines and validated our custom architectural decisions.

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL*, 2019.

[2] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv:1907.11692*, 2019.

[3] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *JMLR*, 2020.

[4] P. Rajpurkar et al., "SQuAD: 100,000+ Questions for Machine Comprehension of Text," *EMNLP*, 2016.